

Multiverse Superposition Inference: Deferred Collapse over Externally-Weighted Decoding Paths

Authors: Weslyn Cory Whitehead Jr.¹

¹ AsAManThinks / MaiiaM Alchemist Correspondence: yarethewatchman@gmail.com

Preprint version: v1.0 **Date:** 2026-05-13

Abstract

We describe **Multiverse Superposition Inference (MSI)**: a decoding strategy in which a language model maintains K parallel hypothesis trajectories (“multiverse”) during generation, assigns each trajectory an evolving scalar credence (“superposition” weight), and collapses the trajectory bundle to a single output only at *semantic boundaries* — sentence breaks, paragraph breaks, or detected register shifts — rather than at every token.

MSI generalizes three existing ideas. Like diverse beam search [Vijayakumar 2018], it keeps a set of partial hypotheses with explicit diversity pressure. Like self-consistency [Wang 2022], it aggregates across paths before returning a final answer. Like tree-of-thoughts [Yao 2023], it admits external evaluators at branching decisions. The novel contribution is the combination of (a) **deferred collapse** at content-defined boundaries rather than per-step pruning, and (b) **externally-supplied credence signals** — archetype coherence, RAG alignment, register adherence — fused with model logprobs into a unified weighting law.

We frame the algorithm classically. The “quantum” terminology is evocative naming for ordinary weighted-ensemble operations; we make no claims about quantum-mechanical computation. We give the algorithm, discuss KV-cache management (shared-prefix copy-on-write to avoid $K \times$ memory cost), analyze the connections and distinctions versus prior decoding strategies, and document a partial reference implementation in the MaiiaM Alchemist project.

Code status. The current implementation (`packages/training-pipeline/training_pipeline/multiverse/`) realizes the *adapter-level* form of MSI: K LoRA adapters trained from different seeds, each scored against a HeartScale signature, blended at inference time. The *token-level* form described in Sections 3–4 is proposed; we describe what exists and what is open.

1. Introduction

Autoregressive decoding from a transformer language model is, by default, a one-dimensional walk: at each step, the model produces a distribution over the next token, and a sampler — greedy, top- k , nucleus [Holtzman 2020], or temperature-scaled softmax — collapses that distribution to a single emitted token. Beam search and its variants [Vijayakumar 2018] maintain a width- K frontier for the duration of generation, but the frontier is pruned at every step under a single scoring criterion (cumulative logprob, optionally with diversity penalty), and the *final* answer is whichever beam scores highest at end-of-sequence.

Two more recent threads relax the single-path assumption further. **Self-consistency** [Wang 2022] generates K fully independent samples (typically chain-of-thought reasoning traces) and majority-votes on the final answer; the samples never interact during generation. **Tree of Thoughts** [Yao 2023] formalizes search over intermediate reasoning states with explicit evaluators at branch points, but the search is structured (and expensive) — every state expansion is evaluated by an LLM call.

The space between these — *interacting* parallel paths whose lifespan extends beyond the token and whose interaction is governed by externally-supplied signals, not just logprobs — has not, to our

knowledge, been formalized as a decoding strategy in its own right. We call this space **Multiverse Superposition Inference** and describe one concrete instance of it.

The intuition. A skilled writer composes a sentence holding two or three candidate phrasings in mind simultaneously, evaluating each against a constellation of constraints — does it sound right, does it follow the argument, is it the same *register* as the surrounding paragraph — and commits only at the sentence break. Standard decoding does not do this: it commits at every token, and any “considering alternatives” must happen inside the next-token distribution, which is too local. MSI moves the commit point out to where the writer would commit, and lets the credence weights be informed by content-dependent signals that the next-token softmax cannot see.

Contributions.

1. **Algorithmic.** A decoding strategy that maintains K paths between *collapse points*, with credence weights that update from both logprob and external signals (Section 3).
2. **System.** A KV-cache discipline (shared-prefix copy-on-write) that keeps memory cost in $O(\text{prefix} + K \times \text{suffix})$ rather than $O(K \times \text{total})$ (Section 4).
3. **Differentiation.** A precise comparison to diverse beam search, self-consistency, speculative decoding, lookahead decoding, and ToT, distinguishing what MSI does that they do not (Section 5).
4. **Partial implementation.** Reference code at the adapter level (Section 6); pseudocode for the token-level form (Appendix A).

We are explicit: the **quantum framing is metaphorical**. “Super- position” is a name for a weighted ensemble; “collapse” is a name for argmax / weighted-merge / vote at a boundary. We do not claim quantum-mechanical effects, quantum speedup, or any property specific to physical quantum systems.

2. Background

2.1 Beam search and diverse beam search

Standard beam search [Sutskever 2014; Freitag 2017] maintains the top- K prefixes by cumulative logprob, expanding each by V continuations and pruning back to K at every step. Diverse beam search [Vijayakumar 2018] adds a Hamming-style diversity penalty so the K beams are not near-duplicates, which is a known failure mode. Two properties characterize beam search: pruning is **per-step**, and the scoring is **logprob-only**.

2.2 Sampling

Nucleus / top- p sampling [Holtzman 2020] addressed the “neural text degeneration” pathology of pure-greedy decoding by sampling from the smallest set of tokens whose cumulative probability exceeds p . Sampling is a single-path procedure; it does not consider alternatives.

2.3 Self-consistency

Self-consistency [Wang 2022] runs K fully independent sampled generations of a chain-of-thought prompt and returns the **majority- vote answer**. It is, in effect, ensemble inference where the ensemble members are independent rollouts of the same model. The key properties: paths **never interact**, paths cannot share KV cache, scoring is a **single vote at the end** rather than during generation.

This is the prior strategy closest to MSI; Section 5.1 differentiates in detail.

2.4 Speculative and assisted decoding

Speculative decoding [Leviathan 2023] uses a small *drafter* model to propose k tokens ahead which the large *verifier* model accepts or rejects in a single forward pass. Assisted generation [Chen 2023] is the same idea with slight variations. **One emitted sequence**, accelerated by parallel verification of a single path.

Lookahead decoding [Fu 2024] removes the drafter by running n -gram guesses through the verifier’s parallel attention, again accelerating a single path. These are *speedup* methods, not *quality* methods, and they do not maintain multiple final-output candidates.

2.5 Tree of Thoughts and Skeleton of Thought

Tree of Thoughts [Yao 2023] formalizes deliberate search at the *thought* level: at each step, the LLM proposes several candidate next thoughts, an LLM-judge evaluates each, and a search algorithm (BFS/DFS with explicit pruning) explores the tree. Skeleton of Thought [Ning 2023] decomposes generation into a skeleton plus parallel section-fill, primarily for latency. ToT and its variants are *deliberate-reasoning* methods; the cost is dominated by judge calls.

2.6 Verifiers

GSM8K [Cobbe 2021] introduced the use of a *trained verifier* to re-rank candidate completions on math word problems. The verifier is an external scorer applied **after** generation completes; it does not influence trajectory selection mid-generation.

For a broader and recent landscape, see Welleck et al.’s survey of LLM decoding [Welleck 2024].

3. Multiverse Superposition Inference

3.1 State

At any decoding time t , MSI maintains a multiset of *paths*:

$$\mathcal{M}_t = \{(s_t^{(1)}, c_t^{(1)}, K_t^{(1)}), \dots, (s_t^{(K)}, c_t^{(K)}, K_t^{(K)})\}$$

where each path i carries:

- $s_t^{(i)}$ — the token sequence emitted by path i up to time t .
- $c_t^{(i)} \in [0, 1]$ — the **credence** of path i , with $\sum_i c_t^{(i)} = 1$.
- $K_t^{(i)}$ — the path’s KV-cache tail (Section 4 discusses sharing).

We use the name *credence* (following Bayesian terminology) rather than “amplitude” to keep the framing classical: it is a probability weight, not a complex amplitude.

3.2 Collapse points

A **collapse point** is a content-defined location at which the multiverse contracts to a single emitted sequence. We use three heuristics, any of which fires:

- **Sentence boundary**: emission of $.$, $!$, $?$, or paragraph break.
- **Register shift**: a change in the dominant Vortex archetype (Section 6) that exceeds a threshold.
- **Length cap**: a maximum tokens-since-last-collapse to bound worst-case memory.

Collapse points are detected after a token is emitted by each path *independently*; the collapse procedure operates over the bundle as soon as a majority of paths (weighted by credence) have reached a collapse point. Paths that have not yet reached one are extended greedily a bounded number of steps to align.

3.3 Credence update

Between collapse points, the credence vector $\mathbf{c}_t$ updates multiplicatively at every token step from three sources:

$$c_{t+1}^{(i)} \propto c_t^{(i)} \cdot \pi(x_t^{(i)} \mid s_{<t}^{(i)}) \cdot \exp\left(\sum_j \lambda_j \cdot \phi_j(s_{\leq t}^{(i)})\right)$$

normalized so $\sum_i c_{t+1}^{(i)} = 1$, where:

- $\pi(x_t^{(i)} \mid s_{<t}^{(i)})$ — the model’s logprob of the token just emitted on path i . This is the *intrinsic* signal (beam search uses only this).
- $\phi_j(\cdot)$ — the j -th **external signal**, a scalar function of the partial sequence. Values near 0 are neutral; positive values reward, negative values penalize. We use three signals in our reference design (Section 6):
 - **Archetype coherence**: agreement between the path’s Vortex archetype trace and a target archetype profile.
 - **RAG alignment**: cosine similarity between path tokens and the retrieved corpus.
 - **Register adherence**: agreement with the prompt’s Wings register classification.
- λ_j — per-signal temperature, controlling its influence relative to logprob.

The multiplicative-in-log form is the standard log-linear ensemble [Och 2003] applied per-step. With $\lambda_j = 0$ for all j , MSI reduces to diverse beam search with a softmax-on-logprob credence (rather than top- K pruning).

A few practical notes on the credence update. First, the per-step update is numerically delicate: multiplying probabilities at every step underflows quickly. We carry log-credence $\ell_t^{(i)} = \log c_t^{(i)}$ throughout, normalize via log-sum-exp at each step, and materialize $c_t^{(i)} = \exp(\ell_t^{(i)})$ only when needed (for collapse decisions). Second, the external signals ϕ_j are typically *much* slower-varying than the per-token logprob: cosine similarity to a retrieved chunk does not change meaningfully token- to-token. We therefore cache ϕ_j values and recompute them only every Δ tokens (we use $\Delta = 4$ in the reference design), interpolating linearly in between. This brings per-token overhead down to a few percent of model forward cost. Third, when a signal ϕ_j is undefined (e.g., RAG alignment for a generation with empty retrieval), we set its contribution to zero rather than penalize; this is the equivalent of an uninformative prior on that signal.

The credence vector additionally serves a *diagnostic* role: a collapsed distribution (one path holding > 0.9 credence) indicates the multiverse has effectively become single-path, and the runtime can short-circuit the remaining $K - 1$ forward passes for the rest of the segment. A flat distribution (max credence near $1/K$) indicates the model genuinely is uncertain across phrasings, and the collapse-point decision will matter. We log credence entropy per token as a generation-time observability signal.

3.4 The collapse procedure

At a collapse point with credence vector $\mathbf{c}$ and emitted sequences $\{s^{(i)}\}$, the collapse operator $\mathcal{C}$ returns a single sequence s^* and a fresh multiverse seeded from s^* . We define three collapse modes:

Argmax collapse (deterministic). $s^* = s^{(i^*)}$ where $i^* = \arg \max_i c^{(i)}$. Simplest, equivalent to “pick the best beam at the sentence break.”

Sampled collapse (stochastic). s^* is sampled from $\mathbf{c}$. Preserves diversity across runs; useful for creative generation.

Token-vote collapse (positionally aware). For each token position p in the segment-since-last-collapse, vote across paths weighted by credence. Functionally similar to self-consistency applied locally; useful when paths agree on most tokens and differ on a few.

After collapse, the multiverse is **re-spawned** from s^* : K paths are initialized with shared prefix and divergent next-token samples, KV-cache shared via copy-on-write (Section 4).

3.4.1 Worked example

Consider generating the second sentence of a passage. Suppose $K = 3$ paths have emitted:

- Path A: "The dawn arrived slowly," — credence 0.42, archetype trace dominated by *Tide* (E=1, T=0).
- Path B: "At sunrise, the analyst noted" — credence 0.31, archetype trace dominated by *Compass* (T=1, R=1).
- Path C: "Morning came; she felt" — credence 0.27, archetype trace dominated by *Bloom* (T=1, E=1).

Suppose the prompt’s target archetype profile (from the system context) is *Bloom*. The archetype-coherence signal ϕ_1 strongly rewards path C; the logprob signal modestly favors path A (more common phrasings). The per-step multiplicative update over the next few tokens drives path C’s credence up at the expense of A and B.

At the sentence break (collapse point), suppose credences have become A=0.18, B=0.22, C=0.60. Under **argmax collapse**, the committed sentence is path C’s completion. Under **sampled collapse**, path C is chosen with probability 0.60. Under **token-vote collapse**, each position votes, and (since the paths agree on no shared tokens here) the result is dominated by the highest-credence path.

After collapse, the cache `shared_kv` is extended with path C’s emitted tokens, and the multiverse re-spawns three new paths from that new prefix. The next sentence’s “considered alternatives” are fresh; the model is not locked into path C’s stylistic choices for the rest of the generation, only for the sentence just committed.

3.5 Path spawning

Initial paths at $t = 0$ are spawned by sampling K distinct first-tokens from the model’s distribution under temperature τ_{spawn} . We deduplicate paths that agree on the first m tokens to maintain diversity.

After each collapse, the same procedure runs from s^* . The total number of distinct *trajectories* considered across the full generation is $K \times C$ where C is the number of collapse points — far larger than K alone.

4. KV-Cache Management

Naïve MSI would multiply KV-cache memory by K , since each path needs its own attention state. This is unacceptable: a 70B-class model at 4K context already strains memory; $K = 4$ would blow it.

We use **shared-prefix copy-on-write** (SPCOW) on the KV cache. The discipline:

- The cache is structured as an immutable shared prefix plus K divergent tails.
- Between collapse points, paths write only to their own tail; the shared prefix is never modified.
- At a collapse point, the surviving sequence s^* 's tail becomes part of the new shared prefix. The K tails for the next segment are again divergent.

Memory cost: $O(L_{\text{prefix}} + K \cdot L_{\text{segment}})$ where L_{segment} is the typical tokens-between-collapse (roughly one sentence, ~ 20 -30 tokens). For $K = 4$ and 30-token segments, the overhead is ≈ 120 tokens of cache — negligible against a 4K context.

This is similar to the cache management already used in production inference servers for sibling-request batching (e.g., vLLM's PagedAttention [Kwon 2023]); MSI repurposes the same mechanism for within-request siblings rather than across-request.

4.1 Compute cost

The K paths share compute over the prefix (one forward pass) and diverge over the segments (K forward passes per segment token). Total compute is:

$$C_{\text{MSI}} = C_{\text{prefix}} + K \cdot C_{\text{suffix}}$$

vs. self-consistency's $K \cdot C_{\text{total}}$. For typical prompts where prefix $\gg$ suffix, MSI is substantially cheaper than self-consistency at the same K .

4.2 Compatibility with speculative decoding

MSI is orthogonal to speculative decoding [Leviathan 2023]: each path can independently use a drafter+verifier pair. The credence update is unchanged.

5. Differentiation from Prior Work

5.1 vs. Self-consistency

Self-consistency [Wang 2022] is the closest prior. Differences:

Property	Self-consistency	MSI
Path interaction	None (fully independent)	Credence-weighted across paths
KV-cache sharing	No	Yes (shared prefix)
Collapse frequency	Once, at end of generation	Every collapse point ($\sim$ sentence)
External signals	None during generation	Per-step, multi-signal
Aggregation	Majority vote on final answer	Multiple modes (argmax/sample/per-token vote)
Compute	$K \cdot C_{\text{total}}$	$C_{\text{prefix}} + K \cdot C_{\text{suffix}}$

Self-consistency is a *post-hoc* ensemble; MSI is an *online* ensemble with mid-generation interaction. The two are not exclusive — one could run N MSI generations and self-consistency-vote across them.

5.2 vs. Diverse beam search

Diverse beam search [Vijayakumar 2018]: - Prunes at every step. - Scores logprob-only (plus the diversity penalty). - Returns the top-1 beam at end-of-sequence.

MSI: - Prunes only at collapse points. - Scores logprob + multiple external signals. - Returns the collapse-time winner per segment (so the final sequence is a *sequence of segment-winners*, not one whole-sequence winner).

The “sequence of segment-winners” property is important. Beam search optimizes for a globally high-logprob single sequence; MSI optimizes each segment under content-aware signals that vary across the generation (e.g., the dominant Vortex archetype may shift mid-paragraph; MSI’s credence update tracks this).

5.3 vs. Tree of Thoughts

ToT [Yao 2023] uses explicit search over reasoning *states*, with per-step LLM-judge calls. MSI: - Branches at the **token** level, not the thought level. - Uses **cheap classical signals** (cosine sim, archetype agreement) — not LLM-judge calls — for credence. - Has **no search structure** (no DFS/BFS); it’s a flat K -wide beam.

Cost per token is dominated by K forward passes, not by N judge calls. MSI is closer to “decoding” in spirit; ToT is closer to “search.”

5.4 vs. Speculative / lookahead decoding

Speculative decoding [Leviathan 2023] and lookahead decoding [Fu 2024] accelerate *single-path* generation. MSI deliberately maintains K paths — it is a quality / controllability tool, not a speed tool. The two compose freely.

5.5 vs. Verifier-based re-ranking

Cobbe 2021’s verifiers re-rank fully-generated candidates. MSI applies external scorers *during* generation. The two are complementary: a final verifier pass can re-rank the post-collapse output, just as one would with any decoding strategy.

5.6 vs. Skeleton of Thought

Skeleton of Thought [Ning 2023] is fundamentally a *latency* tool (parallel section-fill). MSI has no parallelism across logical sections; its parallelism is across alternative phrasings of the same span.

6. Reference Implementation

The MaiaM Alchemist project contains a partial reference implementation under `packages/training-pipeline/t`. We distinguish three levels of MSI, only the first of which is currently shipped:

6.1 Adapter-level MSI (shipped)

In the shipped form, “paths” are not token-level trajectories but **LoRA adapters trained from different seeds**. Specifically:

- `parallel_train.MultiverseTrainOrchestrator` trains N LoRA adapters from `base_seed + i` over the same base model and same corpus. On MPS, this is strictly sequential (one adapter resident at a time); on CUDA with sufficient VRAM, it parallelizes.
- `adapter_bank.AdapterBank` catalogs each adapter and stores a **HeartScale signature**: the mean of the corpus’s feature vectors in HeartScale space.
- `heartscale_blender.HeartScaleBlender` maps a prompt to per-adapter weights by cosine-similarity between the prompt’s HeartScale features and each adapter’s signature, then softmaxes (or argmaxes, or uniforms).
- `sidecar/multiverse_endpoint.py` exposes the blender over RPC as `alchemist.multiverse.predict`.

This is MSI with $K = N$ paths where the “path” is the entire generation pass through a single adapter, the “credence” is the HeartScale softmax weight, and “collapse” happens once at the end (picking the highest-weight adapter, or — proposed — blending logits across all adapters during generation). The shipped endpoint currently picks the winner; full logit-blending is documented but stubbed (`forward_fn` is injectable).

This is closer in spirit to ensemble-of-experts inference than to token-level multiverse, but the algebraic structure — credence vector, softmax-by-affinity, weighted blend — is identical.

6.2 Segment-level MSI (proposed)

A natural intermediate: re-run the blender at every sentence boundary, allowing the active-adapter weight distribution to shift mid-generation as the partial output’s HeartScale features evolve. The KV-cache discipline of Section 4 applies. This is implementable today on top of the existing adapter bank; it is on the roadmap.

6.3 Token-level MSI (proposed)

The full form of Section 3: K paths in *one* model, with credence update at every token and collapse at sentence breaks. This requires (a) a custom decoding loop with SPCOW cache, (b) the external scorer hooks wired into the loop, (c) a collapse-point detector. The math is in this paper; the code is not yet written.

6.4 External signal sources

The platform supplies three signal candidates:

- **Archetype coherence** — agreement between the running Vortex archetype assignment of the partial sequence and the prompt’s target archetype (see the companion *Vortex-Keyed MoE Routing* preprint for the archetype framework).
- **HeartScale / register adherence** — cosine similarity in the Wings register feature space, as already wired in the adapter-level blender.
- **RAG alignment** — cosine similarity between the partial sequence’s embedding and the retrieved corpus chunks for the query.

These are cheap (sub-millisecond) per call and additive in the log-credence update.

7. Properties and Caveats

7.1 When MSI is expected to help

- **Multi-register generation.** When the desired register changes mid-output (e.g., from analytical to lyrical), MSI’s per-segment scoring tracks the shift; beam search and self-consistency, which score the whole sequence under one criterion, do not.
- **Constraint-conditioned generation.** When an external signal (RAG alignment, style classifier) is available, MSI uses it online rather than as a post-hoc filter.
- **Tasks with sentence-level coherence requirements.** Argument- building, multi-step reasoning where each step must be locally defensible.

7.2 When MSI is unlikely to help

- **Short outputs** (single sentence, classification). No collapse points to exploit; reduces to self-consistency or beam.
- **Tasks where logprob is sufficient.** Code generation with a unit-test verifier: a single verifier pass at the end is cheaper than per-token external scoring.
- **Memory-constrained deployments where SPCOW is not available.** Without shared-prefix cache, K paths cost $K \times$ memory, which may exceed budget.

7.3 Open questions

- **What K is optimal?** We expect a sweet spot around $K = 3\text{--}8$ based on self-consistency results [Wang 2022], but it is not measured for MSI specifically.
- **How should λ_j be set?** Per-signal temperatures trade off intrinsic plausibility (logprob) against external guidance. Hand-tuning is feasible; learned schedules are open.
- **Collapse-point detection failures.** A bad sentence detector forces argmax collapse on disjoint mid-thought sequences, producing incoherent splices. Detector quality matters.
- **Diversity collapse.** Like beam search, MSI can suffer when all K paths converge to near-duplicates. The diversity-penalty trick from Vijayakumar 2018 applies; we recommend it.

7.4 What this work does not claim

We claim **no quantum-mechanical effect**. The names *superposition* and *collapse* are evocative metaphors for weighted ensembles and weighted aggregation. The algorithm is fully classical, runs on classical hardware, and exhibits no Bell-inequality violations or related quantum signatures. Authors who object to the terminology may freely substitute *multi-candidate inference with deferred aggregation*; the math is the same.

We claim **no headline benchmark numbers** in this preprint. The adapter-level form is shipping; the token-level form is described algorithmically and remains to be empirically validated. v1.1 will report results.

8. Conclusion

We described Multiverse Superposition Inference: a decoding strategy that maintains K parallel hypothesis paths with evolving credence weights, fuses model logprobs with external content-aware signals, and collapses to a single output at semantic boundaries rather than at every step. Memory cost is controlled by a shared- prefix copy-on-write KV-cache discipline. The strategy generalizes

diverse beam search, complements self-consistency, and provides a clean place to insert content-aware credence signals (RAG alignment, register coherence, archetype tracking) without rebuilding the model.

The current implementation in the MaiaM Alchemist project realizes the *adapter-level* form of MSI. The token-level form, with full collapse-point machinery, is specified in this paper and is on the roadmap. The contribution is the algorithm and the framing; the empirical case is open.

References

- Chen, C. et al. (2023). *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv:2302.01318.
- Cobbe, K. et al. (2021). *Training Verifiers to Solve Math Word Problems*. arXiv:2110.14168.
- Freitag, M., Al-Onaizan, Y. (2017). *Beam Search Strategies for Neural Machine Translation*. arXiv:1702.01806.
- Fu, Y., Bailis, P., Stoica, I., Zhang, H. (2024). *Break the Sequential Dependency of LLM Inference Using Lookahead Decoding*. ICML.
- Holtzman, A., Buys, J., Du, L., Forbes, M., Choi, Y. (2020). *The Curious Case of Neural Text Degeneration*. ICLR.
- Kwon, W. et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP.
- Leviathan, Y., Kalman, M., Matias, Y. (2023). *Fast Inference from Transformers via Speculative Decoding*. ICML.
- Ning, X. et al. (2023). *Skeleton-of-Thought: Large Language Models Can Do Parallel Decoding*. arXiv:2307.15337.
- Och, F. J. (2003). *Minimum Error Rate Training in Statistical Machine Translation*. ACL.
- Sutskever, I., Vinyals, O., Le, Q. (2014). *Sequence to Sequence Learning with Neural Networks*. NeurIPS.
- Vijayakumar, A. K. et al. (2018). *Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models*. AAAI.
- Wang, X. et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. arXiv:2203.11171.
- Welleck, S. et al. (2024). *From Decoding to Meta-Generation: Inference-time Algorithms for Large Language Models*. TMLR survey.
- Yao, S. et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS.

Appendix A — Algorithm Pseudocode

```
def msi_decode(model, prompt, K=4, signals=(), lambdas=(),
               max_tokens=1024, collapse_mode="argmax",
               spawn_temp=1.0):
    """
    Multiverse Superposition Inference.
```

```

model:      autoregressive LM with .forward(tokens, kv_cache)
prompt:     input token ids
K:          multiverse width
signals:    list of scorer fns: partial_seq -> float
lambdas:    list of per-signal temperatures, same len as signals
collapse_mode: 'argmax' | 'sample' | 'token_vote'
"""

# 1. Prefill shared KV cache from prompt.
shared_kv = model.prefill(prompt)

# 2. Spawn K paths with divergent first tokens.
first_logits = model.next_token_logits(shared_kv)
first_toks = sample_distinct(first_logits, K, temp=spawn_temp)
paths = [
    Path(seq=[t], kv_tail=clone_tail(shared_kv, t),
         credence=1.0/K, alive=True)
    for t in first_toks
]

out = [] # committed (post-collapse) tokens

while sum_len(out) + max(len(p.seq) for p in paths) < max_tokens:
    # 3. Step each path one token.
    for p in paths:
        if not p.alive:
            continue
        logits = model.next_token_logits_with_tail(
            shared_kv, p.kv_tail)
        tok = sample(logits) # nucleus / temperature / etc.
        p.seq.append(tok)
        p.kv_tail = append_to_tail(p.kv_tail, tok, logits)

    # 4. Credence update: logprob  $\times \exp(\sum \lambda_j \phi_j)$ .
    logp = log_softmax(logits)[tok]
    ext = sum(l * phi(prompt + out + p.seq)
              for l, phi in zip(lambdas, signals))
    p.credence *= exp(logp + ext)

    # 5. Has this path hit a collapse point?
    if is_collapse_point(p.seq):
        p.alive = False # waiting for the rest

normalize_credences(paths)

# 6. If a credence-weighted majority has collapse-pointed,
#     collapse the bundle.
weighted_done = sum(p.credence for p in paths if not p.alive)
if weighted_done >= 0.5:

```

```

    committed_seq = collapse(paths, mode=collapse_mode)
    out.extend(committed_seq)

    # 7. Re-spawn from the committed sequence.
    shared_kv = extend_kv(shared_kv, committed_seq)
    first_logits = model.next_token_logits(shared_kv)
    first_toks = sample_distinct(first_logits, K,
                                temp=spawn_temp)

    paths = [
        Path(seq=[t], kv_tail=clone_tail(shared_kv, t),
            credence=1.0/K, alive=True)
        for t in first_toks
    ]

    if is_eos(committed_seq):
        break

return out

def collapse(paths, mode):
    if mode == "argmax":
        winner = max(paths, key=lambda p: p.credence)
        return winner.seq
    if mode == "sample":
        return sample_by_weight(paths,
                                weights=[p.credence for p in paths]).seq
    if mode == "token_vote":
        # Align by position; weighted vote per position.
        # Assumes paths agree on length (truncate to min).
        L = min(len(p.seq) for p in paths)
        out = []
        for pos in range(L):
            votes = defaultdict(float)
            for p in paths:
                votes[p.seq[pos]] += p.credence
            out.append(max(votes.items(), key=lambda kv: kv[1])[0])
        return out

```

KV-cache sharing detail

`shared_kv` holds the K-V tensors for tokens shared by all paths. `p.kv_tail` holds *only* the tail tokens emitted by path `p` since the last collapse. The model’s attention mechanism is called with the concatenation (`shared_kv`, `p.kv_tail`), which on production serving stacks (e.g., vLLM’s PagedAttention [Kwon 2023]) is a single block-table operation rather than a copy.

At collapse, the winner’s `kv_tail` is appended to `shared_kv` and the new `kv_tails` are reset to empty before re-spawning. Memory cost across the multiverse is therefore *one* shared prefix plus K short tails, not K full caches.

Appendix B — Reference Implementation Pointers

Adapter-level MSI (shipped in MaiiaM Alchemist):

- Orchestrator (outer N-seed loop): `maiiam-alchemist/packages/training-pipeline/training_pipeline.py` — `MultiverseTrainOrchestrator`, `MultiverseConfig`.
- HeartScale blender (credence assignment per adapter): `.../multiverse/heartscale_blender.py` — `HeartScaleBlender`, `heartscale_softmax`, `winner_takes_all`, `uniform`, `BlendResult`.
- Adapter catalog and signatures: `.../multiverse/adapter_bank.py` — `AdapterBank`, `AdapterEntry`, `AdapterSignature`.
- RPC endpoint: `maiiam-alchemist/packages/harmonic-engine/harmonic_engine/sidecar/multiverse.py` — `alchemist.multiverse.predict`.

Token-level MSI (Section 3, Appendix A): not yet implemented. Specified for v1.1.

End of preprint v1.0.